CO5

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar **DATE:19.08.2026**

REG.NO: 192411119 **Wednesday**

COURSE CODE : CSA1103

COURSE NAME : OOAD

SECTION A – SELF INTRODUCTION

1. Tell me about yourself.

Answer:

Good morning, sir/madam. My name is **P. Ranjithkumar**, and my ID is **192411119**. I am pursuing my B.Tech in Software Engineering. I am interested in software development, programming, and software design. I have learned programming languages such as Python, C++, and Java and studied subjects such as Object-Oriented Analysis and Design. In OOAD, I have learned classes, objects, inheritance, polymorphism, encapsulation, abstraction, and UML diagrams. I am a hardworking and quick-learning person. My goal is to improve my technical skills and become a successful software developer.

2. Why did you choose Computer Science?

Answer:

I chose Computer Science because I am interested in technology, programming, and problem-solving. Computer Science provides many opportunities to develop applications and solve real-world problems using technology. I also enjoy learning programming languages and developing software solutions.

3. Explain your favorite subject in OOAD.

Answer:

My favorite topic in OOAD is **UML modeling**. UML helps us visually represent the structure and behavior of a software system. Class diagrams, use case diagrams, sequence diagrams, and activity diagrams make complex systems easier to understand and design.

4. What software development tools have you used?

Answer:

I have used tools such as **VS Code, Git, GitHub, MySQL, Google Colab, and UML modeling tools**. I use VS Code for programming, Git and GitHub for version control, MySQL for database management, and UML tools for software modeling.

5. Describe a project where you applied object-oriented concepts.

Answer:

In an academic project, I divided the system into different classes according to their responsibilities. I used encapsulation to protect data, inheritance to reuse common properties, and polymorphism to provide different implementations of similar operations. UML diagrams were also useful for representing the structure and interaction of the system.

SECTION B – TECHNICAL QUESTIONS

Q1. What is Object-Oriented Analysis and Design?

Answer:

Object-Oriented Analysis and Design, or **OOAD**, is a software engineering approach that analyzes and designs a system using objects and classes.

Analysis identifies what the system needs to do, while **Design** determines how those requirements will be implemented.

Requirements
↓
Object-Oriented Analysis
↓
Classes + Objects + Relationships
↓
Object-Oriented Design
↓
Implementation

Benefits:

- Reusability
- Maintainability
- Modularity

- Flexibility
- Easier system understanding

Q2. Differentiate between Analysis and Design.

Analysis	**Design**
Determines what the system should do	Determines how the system will work
Focuses on requirements	Focuses on implementation structure
Identifies objects and responsibilities	Defines classes, methods and interfaces
Mostly technology independent	More technology dependent
Produces analysis models	Produces design models

Short interview answer:

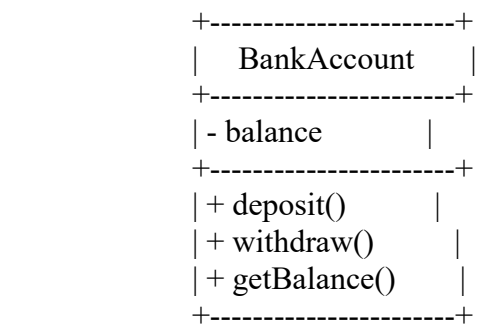
> Analysis focuses on **what**, while design focuses on **how**.

Q3. Explain Encapsulation with an example.

Answer:

Encapsulation means combining data and methods inside a class and restricting direct access to the internal data.

Example:



Here, balance is private. Users cannot directly change it. They must use methods such as deposit() or withdraw().

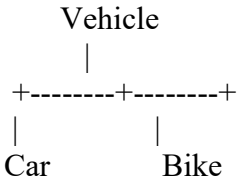
Benefits:

- Data security
- Controlled access
- Better maintainability

Q4. What is Inheritance? Mention its types.

Answer:

Inheritance is an OOP mechanism in which a child class acquires properties and methods from a parent class.



Types:

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance
5. Hybrid inheritance

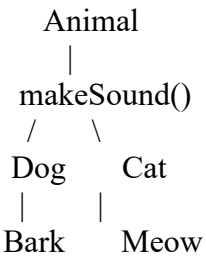
Main benefit: Code reuse.

Q5. Define Polymorphism and its benefits.

Answer:

Polymorphism means **one interface or method can have different implementations**.

Example:



Both Dog and Cat implement makeSound() differently.

Benefits

- Flexibility

- Reusability
- Extensibility
- Easy maintenance
- Supports loose coupling

Q6. What is Abstraction?

Answer:

Abstraction means showing only the essential features of an object while hiding unnecessary implementation details.

Example:

When we use an ATM, we select:

ATM
|
+-- Withdraw
|
+-- Deposit
|
+-- Balance

We do not need to know how the internal banking system processes the transaction.

Benefit: Abstraction reduces complexity.

Q7. Difference between Aggregation and Composition.

Aggregation

Aggregation is a **weak has-a relationship**. The child object can exist independently.

Department ◇——— Teacher

A teacher can exist even if the department is removed.

Composition

Composition is a **strong has-a relationship**. The child depends on the parent.

House ◆——— Room

Difference

Aggregation	Composition
Weak relationship	Strong relationship
Independent lifetime	Dependent lifetime
Hollow diamond	Filled diamond

Q8. What is Coupling and Cohesion?

Coupling

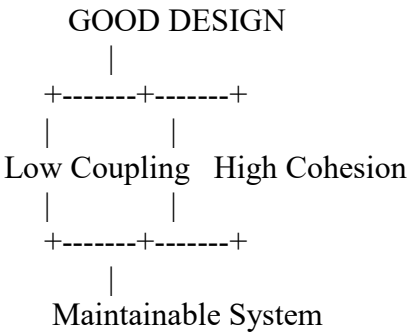
Coupling represents the dependency between different modules or classes.

Low coupling is preferred.

Cohesion

Cohesion represents how closely related the responsibilities inside a module are.

High cohesion is preferred.



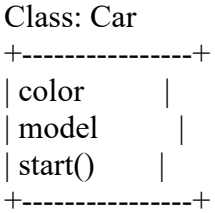
Q9. What is a Class and an Object?

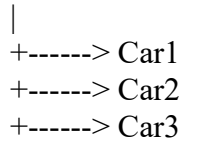
Class

A class is a blueprint or template for creating objects.

Object

An object is an instance of a class.





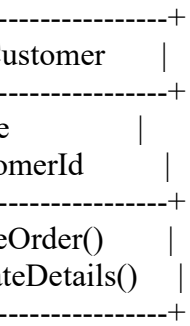
For example, Car is a class, while a particular red car is an object.

Q10. Explain Object Responsibility.

Answer:

Object responsibility defines what an object should **know and do**.

For example:



The Customer object is responsible for maintaining customer information and performing customer-related operations.

Proper responsibility assignment improves **cohesion and maintainability**.

SECTION C – UML MODELING

Q11. What is UML?

Answer:

UML stands for **Unified Modeling Language**. It is a standard visual language used to specify, visualize, construct, and document software systems.

Common UML diagrams include:

- Use Case Diagram
- Class Diagram
- Sequence Diagram

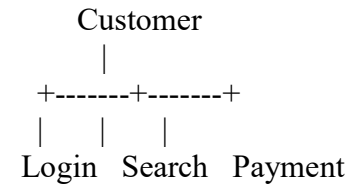
- Activity Diagram
- State Diagram
- Component Diagram
- Deployment Diagram

Q12. When would you use a Use Case Diagram?

Answer:

A Use Case Diagram is mainly used during requirements analysis. It shows the **actors interacting with the system and the functions they perform**.

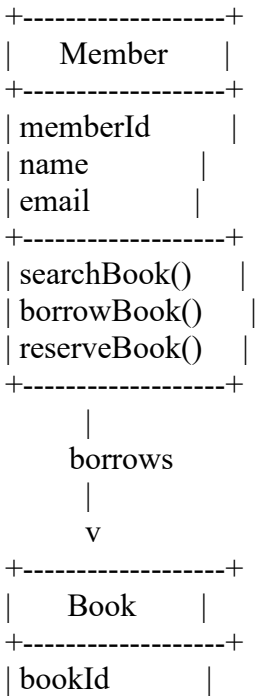
Example:

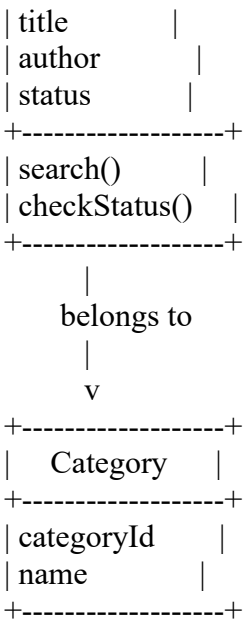


It helps developers and customers understand system requirements.

Q13. Draw and explain a Class Diagram for a Library Management System.

Class Diagram





Explanation

The main classes are:

- Member
- Book
- Category

A member can borrow or reserve books. Each book can belong to a category. The diagram represents classes, attributes, methods, and relationships.

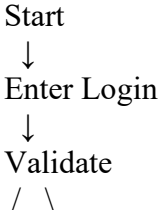
Q14. Differentiate Sequence Diagram and Activity Diagram.

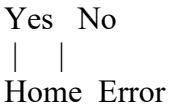
Sequence Diagram	**Activity Diagram**
Shows object interactions	Shows workflow
Focuses on message sequence	Focuses on activities
Shows time/order of messages	Shows process flow
Useful for a particular scenario	Useful for business processes

Sequence Example

User → Login → Authentication → Database

Activity Example



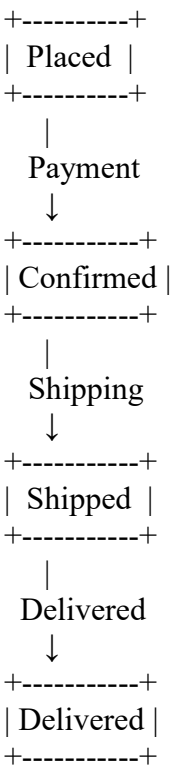


Q15. Explain State Diagram with an example.

Answer:

A State Diagram shows the different states of an object and the transitions between those states.

Example: Online Order



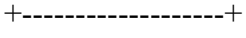
State diagrams are useful for systems where an object changes state based on events.

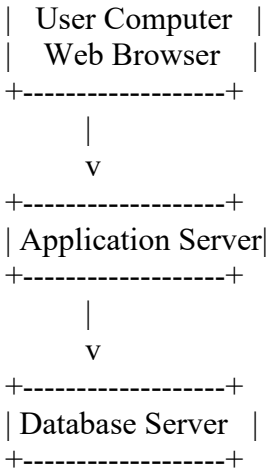
Q16. What is a Deployment Diagram?

Answer:

A Deployment Diagram shows the physical arrangement of hardware nodes and the software deployed on them.

Example



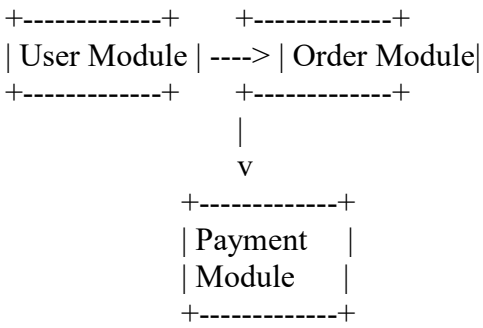


It helps architects understand how software components are deployed on hardware.

Q17. How do Component Diagrams help software architects?

Answer:

A Component Diagram represents the major software components and their dependencies.



Benefits

- Shows system architecture.
- Identifies dependencies.
- Supports modular design.
- Helps developers understand system structure.
- Makes maintenance easier.

Q18. What is Multiplicity in UML?

Answer:

Multiplicity specifies how many objects of one class can be associated with another class.

Common values:

- 1 = Exactly one
- 0..1 = Zero or one
- * = Many
- 1..* = One or more

Example

Customer 1 -------- 0..* Order

This means one customer can have zero or many orders.

SECTION D – CASE STUDY / PROBLEM SOLVING

CASE STUDY 1 – Hospital Management System

The system manages:

- Patients
- Doctors
- Appointments
- Billing

1. Identify Classes

The main classes are:

Patient
Doctor
Appointment
Billing
Hospital

Patient

patientId
name
age
phone
address

Doctor

doctorId
name

specialization

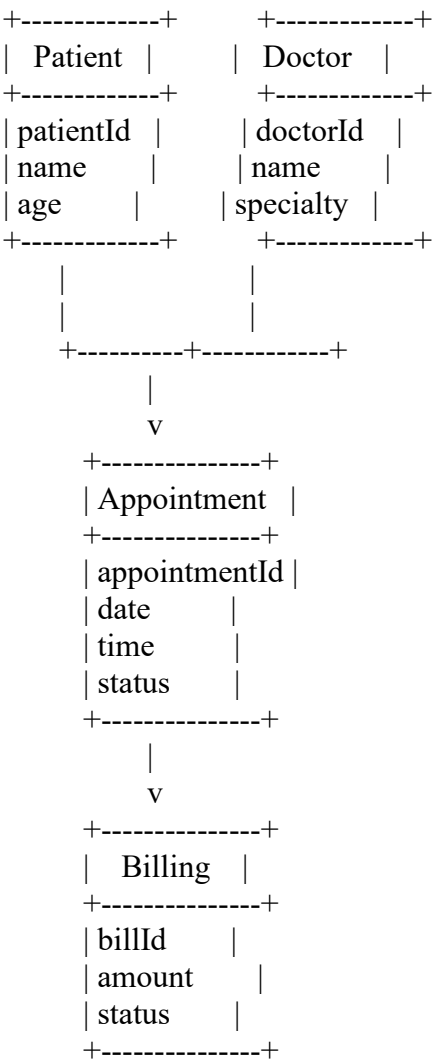
Appointment

appointmentId
date
time
status

Billing

billId
amount
paymentStatus

2. Simple Class Diagram



3. Relationships

- Patient **books** Appointment.
- Doctor **attends** Appointment.

- Appointment **generates** Billing.
- Patient **has** Billing.

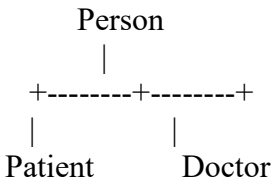
4. UML Diagrams Used

I would use:

1. Use Case Diagram — system requirements.
2. Class Diagram — classes and relationships.
3. Sequence Diagram — appointment booking interaction.
4. Activity Diagram — hospital workflow.
5. State Diagram — appointment status.

5. Inheritance

Inheritance can be applied using a common Person class.



Person may contain common attributes such as:

name
age
phone
address

Patient and Doctor can then have their own specialized attributes and methods.

CASE STUDY 2 – ONLINE SHOPPING SYSTEM

Features:

- Customers
- Products
- Cart
- Orders
- Payments

1. Actors and Use Cases

Actor

Customer

Use Cases

- Register
- Login
- Search Product
- Add Product to Cart
- View Cart
- Place Order
- Make Payment
- Track Order

Use Case Diagram

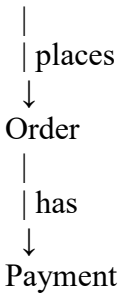
ONLINE SHOPPING SYSTEM

Customer -----> (Register)
Customer -----> (Login)
Customer -----> (Search Product)
Customer -----> (Add to Cart)
Customer -----> (View Cart)
Customer -----> (Place Order)
Customer -----> (Make Payment)
Customer -----> (Track Order)

2. Main Classes and Relationships

Customer
|
| owns
↓
Cart
|
| contains
↓
CartItem
|
| refers to
↓
Product

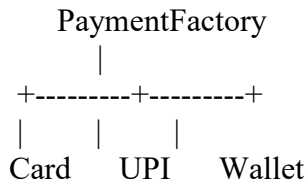
Customer



3. Suitable Design Patterns

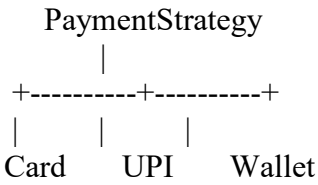
Factory Pattern

Used for creating different payment objects.



Strategy Pattern

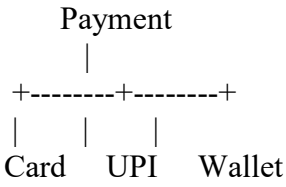
Used when different payment methods need different processing algorithms.



These patterns improve flexibility and make it easier to add new payment methods.

4. Polymorphism in Payment Processing

Create a common payment interface:



Each class implements:

processPayment()

For example:

Payment p

p = CardPayment
p.processPayment()

p = UPIPayment
p.processPayment()

p = WalletPayment
p.processPayment()

The same method call produces different behavior depending on the actual payment object.

Benefit: New payment methods can be added without making major changes to the existing checkout system.

SECTION E – HR & PROFESSIONAL QUESTIONS

Q19. Why should we hire you?

Answer:

> You should hire me because I am hardworking, willing to learn, and interested in software development. I have a basic understanding of programming, OOAD concepts, and software design. I am comfortable learning new technologies and working as part of a team. As a fresher, I am ready to take new responsibilities, improve my skills, and contribute to the organization.

Q20. What are your strengths and weaknesses?

Answer:

> My strengths are that I am a quick learner, hardworking, patient, and responsible. I am also interested in solving technical problems and learning new technologies.
>
> One of my weaknesses is that sometimes I spend more time trying to make my work perfect. I am improving this by prioritizing tasks and managing my time better.

Q21. How do you handle project deadlines?

Answer:

First, I understand the project requirements. Then I divide the project into smaller tasks and prioritize them. I set deadlines for each task and regularly check my progress. If I face a problem, I communicate with my team members or mentor early. This helps me complete the project within the deadline.

Understand Requirements
↓
Divide Tasks
↓
Prioritize
↓
Set Deadlines
↓
Develop
↓
Test
↓
Final Submission

Q22. Have you worked in a team project? Explain your role.

Answer:

Yes, I have worked on academic team projects. My role involved understanding requirements, contributing to the implementation, testing modules, and discussing technical problems with team members. We divided the work according to each member's responsibilities and regularly shared our progress. This helped me improve my communication, teamwork, and problem-solving skills.

Q23. Where do you see yourself in five years?

Answer:

In five years, I see myself as a skilled software professional with strong knowledge of software development, object-oriented design, and system development. I want to gain practical experience by working on real-world projects, improve my technical and communication skills, and take greater responsibilities in my organization. I also want to continuously learn new technologies and contribute to the growth of the company.